

REFACTORTACTICS

Architettura software Unreal Engine 5

Moduli, classi, struttura progetto e milestone Fondazioni

Documento: PDR-03

Versione: 0.1 - Consolidato iniziale

Data: 3 agosto 2026

Baseline tecnica: Unreal Engine 5.8

PDR tecnico per implementazione del vertical slice in UE 5.8.

Controllo del documento

Campo	Valore
Stato	Bozza consolidata per revisione tecnica
Versione	0.1 - Consolidato iniziale
Data	3 agosto 2026
Autore	RefactorTactics / documentazione consolidata con supporto AI
Regola di prevalenza	Decisioni esplicite del progetto > requisiti consolidati > proposte PDR > ricerca web di supporto

Sommario esecutivo

L'architettura separa stato logico, autorità e presentazione. Un game module iniziale contiene domini chiari; C++ governa ciò che è possibile, Data Assets e Blueprint selezionano varianti e presentazione.

Decisioni consolidate

Il resolver non dipende da frame, animazioni o UI. Le celle sono dati compatti; il GameState non ospita intenti privati.

Assunzioni operative

La baseline UE 5.8 e la scelta di mantenere un singolo module sono assunzioni pragmatiche; una patch precisa va bloccata nel repository.

Indice del documento

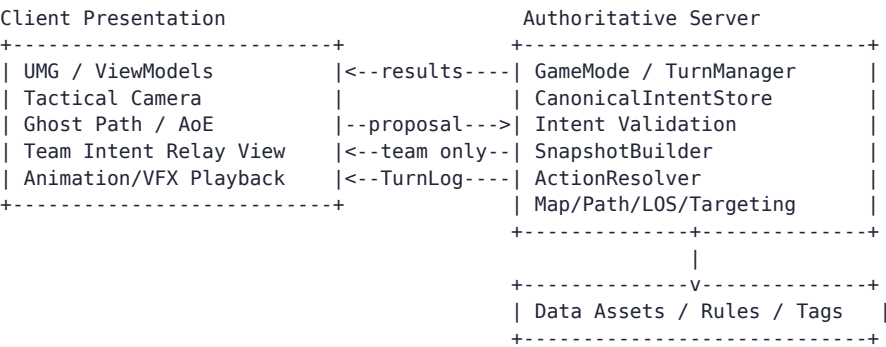
1. Baseline e principi
2. Diagramma architetturale
3. Moduli C++
4. Gameplay Framework
5. Struttura Source
6. Struttura Content
7. Plugin e Build.cs
8. Dipendenze
9. Setup Fondazioni
10. Commit proposto

1. Baseline e principi

Baseline documentale: Unreal Engine 5.8. Il progetto deve bloccare una patch e toolchain esatte nel file README della milestone. La soluzione più semplice scalabile è un singolo game module iniziale, separato internamente per dominio; plugin di progetto solo quando un confine è stabile o riutilizzabile.

- C++: simulazione, rete, serializzazione, pathfinding, validazione, regole competitive e test core.
- Blueprint/Data: configurazione, UI, animazioni, VFX, prototipi e varianti di contenuto.
- GAS: abilità, costi, cooldown, attributi ed effetti; il resolver resta autorità.
- Actor: rappresentazione, collisioni e interazioni visibili; non una cella Actor per ogni nodo.
- Subsystem/servizi: sistemi senza presenza fisica e con ownership esplicita.

2. Diagramma architetturale



3. Moduli C++

Area	Classi/servizi iniziali	Responsabilità
Framework	ARTGameMode, ARTGameState, ARTPlayerController, ARTPlayerState	Lifecycle match, stato pubblico, RPC ownership, identità team.
Unità	ARTTacticalUnit, URTHealthComponent, URTStatusComponent	Rappresentazione unità e stato runtime.
Mappa	URMapSubsystem, FRTMapState, FRTCellId	Lookup celle, revisioni, occupazione, transizioni.
Query	URTPathfindingService, URTLOSService, URTTargetingService	Servizi separati e testabili.
Planning	URTPanningComponent, FRTIntent, FRTIntentStore	Proposte client e store server.
Turno	URTurnManager, FRTSnapshotBuilder, FRTActionResolver	State machine, snapshot, resolution.
Log	FRTTurnLog, FRTTurnEvent	Replay, debug, explainability.
Dati	URTAbilityDefinition, URTCharacterDefinition, URTRuleset	Contenuti versionati e validabili.

4. Gameplay Framework

Il GameMode esiste solo sul server e guida le regole; il GameState contiene esclusivamente lo stato che può essere replicato a tutti. Il PlayerController è il punto naturale per RPC dirette alla singola connessione, mentre il PlayerState conserva identità e dati pubblici persistenti nel match. Le unità possono essere Pawn o Actor controllati logicamente; nel vertical slice non serve forzare CharacterMovement.

5. Struttura Source

```
Source/RefactorTactics/  
  RefactorTactics.Build.cs  
  Public/  
    Core/          RTypes.h, RTIds.h, RRulesVersion.h  
    Framework/     RTGameMode.h, RTGameState.h, RTPlayerController.h  
    Map/           RTCellId.h, RTMapState.h, RTMapSubsystem.h  
    Query/         RTPathfindingService.h, RTLOSSService.h  
    Planning/      RTIntent.h, RTPlanningComponent.h  
    Turn/          RTTurnManager.h, RTSnapshot.h, RTActionResolver.h  
    Unit/          RTTacticalUnit.h, RTStatusComponent.h  
    Data/          RTAbilityDefinition.h, RTCharacterDefinition.h  
    Log/           RTTurnLog.h  
  Private/  
    ... mirrored implementation folders ...  
  Tests/          RTCellIdTests.cpp, RTPathTests.cpp, RTResolverTests.cpp
```

6. Struttura Content

```
Content/RefactorTactics/  
  Maps/Dev/L_DevSandbox  
  Blueprints/Framework/  
  Blueprints/Units/  
  Data/Characters/  
  Data/Abilities/  
  Data/Rulesets/  
  Input/IA_*, IMC_Tactical  
  UI/HUD/, UI/Planning/, UI/Debug/  
  Art/Graybox/, Art/Materials/  
  VFX/, Audio/  
  Tests/Functional/
```

7. Plugin e Build.cs

Modulo/plugin	Milestone Fondazioni	Motivo
Core, CoreUObject, Engine, InputCore	Required	Base runtime.
EnhancedInput	Required	Camera e azioni contestuali.
AIModule	Required	FGraphAStar / GraphAStar.h.
GameplayTags	Required	Vocabolario governato.
UMG, Slate, SlateCore	Required	HUD e debug UI.
GameplayAbilities, GameplayTasks	Dopo path/turn proof	GAS entra quando il resolver base è stabile.
FunctionalTesting	Dev/Test	Test di livello.
CommonUI	Dopo proof of concept	Riduce rischio di integrazione prematura.

Build.cs indicativo

```
PublicDependencyModuleNames.AddRange(new[] {  
    "Core", "CoreUObject", "Engine", "InputCore",  
    "EnhancedInput", "AIModule", "GameplayTags"  
});  
PrivateDependencyModuleNames.AddRange(new[] {  
    "UMG", "Slate", "SlateCore"  
});  
// Aggiungere GameplayAbilities e GameplayTasks nella milestone Abilities.
```

8. Dipendenze

Il grafo delle dipendenze deve puntare verso il core logico: Presentation -> Planning/Queries -> Snapshot/Resolver -> Core Types/Data. Il resolver non dipende da UMG, animazioni o asset caricati in modo implicito. I dati runtime vengono materializzati in strutture compatte prima della resolution.

9. Setup Fondazioni

- 1 Creare progetto Games > Blank > C++, Desktop, Maximum Quality, Starter Content off.
- 2 Impostare L_DevSandbox come Editor Startup Map e Game Default Map.
- 3 Creare ARTGameMode e ARTPlayerController; assegnarli tramite World Settings o DefaultGame.ini.
- 4 Abilitare Enhanced Input e creare IA_CameraPan, IA_CameraZoom, IA_CameraRotate, IA_Select, IA_Confirm, IA_Ready.
- 5 Creare graybox 2D generato da dati compatti e un overlay debug delle coordinate.
- 6 Aggiungere due unità, planning movimento locale, Ready simultaneo locale, snapshot e resolver movimento.
- 7 Abilitare Automation/Functional Testing solo nelle configurazioni appropriate e aggiungere test core.

10. Commit proposto

feat(foundation): bootstrap tactical sandbox and deterministic move turn

- Progetto C++ e Build.cs
- L_DevSandbox e camera tattica
- FRTCellId, mappa graybox e primo A*
- Due unità, planning, Ready, snapshot, movement resolver
- TurnLog, debug overlay e Automation Tests

Fonti tecniche verificate: Epic Games, Gameplay Framework in Unreal Engine, UE 5.8.; Epic Games, Enhanced Input in Unreal Engine, UE 5.8.; Epic Games, FGraphAStar C++ API, UE 5.8.